In [1]:
```
# 1. Un-nesting the columns
# a. Un-nest the columns those have cells with multiple comma separated values by
# creating multiple rows
## Netflix Dataset Case Study
## Data Cleaning and Un-nesting Analysis
## Data Cleaning, Un-nesting and Analysis
## This project focuses on cleaning and transforming the Netflix dataset to make it suitable for analysis.
# The process includes handling missing values, un-nesting multi-value columns, and extracting useful features for insights.

## 1.a. Importing Libraries
## We import essential Python libraries required for data analysis, visualization, and manipulation.
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud

pd.set_option('display.max_columns', None)
pd.set_option('display.max_colwidth', None)
pd.set_option('display.width', 5000)
```

In [2]:
```
## 1.b. Loading Dataset
## The Netflix dataset is loaded into a pandas DataFrame for analysis.

df_netflix = pd.read_csv(r'C:\Users\amit.sehrawat\Python_Practise\Datasets\netflix_titles.csv')
df_netflix.head(10)
```

Out[2]:

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020	PG-13	90 min	Documentaries	As her father nears the end of his life, filmmaker Kirsten Johnson stages his death in inventive and comical ways to help them both face the inevitable.
1	s2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thabang Molaba, Dillon Windvogel, Natasha Thahane, Arno Greeff, Xolile Tshabalala, Getmore Sithole, Cindy Mahlangu, Ryle De Morny, Greteli Fincham, Sello Maake Ka-Ncube, Odwa Gwanya, Mekaila Mathys, Sandi Schultz, Duane Williams, Shamilla Miller, Patrick Mofokeng	South Africa	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows, TV Dramas, TV Mysteries	After crossing paths at a party, a Cape Town teen sets out to prove whether a private-school swimming star is her sister who was abducted at birth.
2	s3	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabiha Akkari, Sofia Lesaffre, Salim Kechiouche, Noureddine Farihi, Geert Van Rampelberg, Bakary Diombera	NaN	September 24, 2021	2021	TV-MA	1 Season	Crime TV Shows, International TV Shows, TV Action & Adventure	To protect his family from a powerful drug lord, skilled thief Mehdi and his expert team of robbers are pulled into a violent and deadly turf war.
3	s4	TV Show	Jailbirds New Orleans	NaN	NaN	NaN	September 24, 2021	2021	TV-MA	1 Season	Docuseries, Reality TV	Feuds, flirtations and toilet talk go down among the incarcerated women at the Orleans Justice Center in New Orleans on this gritty reality series.
4	s5	TV Show	Kota Factory	NaN	Mayur More, Jitendra Kumar, Ranjan Raj, Alam Khan, Ahsaas Channa, Revathi Pillai, Urvi Singh, Arun Kumar	India	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows, Romantic TV Shows, TV Comedies	In a city of coaching centers known to train India's finest collegiate minds, an earnest but

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
												unexceptional student and his friends navigate campus life.
5	s6	TV Show	Midnight Mass	Mike Flanagan	Kate Siegel, Zach Gilford, Hamish Linklater, Henry Thomas, Kristin Lehman, Samantha Sloyan, Igby Rigney, Rahul Kohli, Annarah Cymone, Annabeth Gish, Alex Essoe, Rahul Abburi, Matt Biedel, Michael Trucco, Crystal Balint, Louis Oliver	NaN	September 24, 2021	2021	TV-MA	1 Season	TV Dramas, TV Horror, TV Mysteries	The arrival of a charismatic young priest brings glorious miracles, ominous mysteries and renewed religious fervor to a dying town desperate to believe.
6	s7	Movie	My Little Pony: A New Generation	Robert Cullen, José Luis Ucha	Vanessa Hudgens, Kimiko Glenn, James Marsden, Sofia Carson, Liza Koshy, Ken Jeong, Elizabeth Perkins, Jane Krakowski, Michael McKean, Phil LaMarr	NaN	September 24, 2021	2021	PG	91 min	Children & Family Movies	Equestria's divided. But a bright-eyed hero believes Earth Ponies, Pegasi and Unicorns should be pals — and, hoof to heart, she's determined to prove it.
7	s8	Movie	Sankofa	Haile Gerima	Kofi Ghanaba, Oyafunmike Ogunlano, Alexandra Duah, Nick Medley, Mutabaruka, Afemo Omilami, Reggie Carter, Mzuri	United States, Ghana, Burkina Faso, United Kingdom, Germany, Ethiopia	September 24, 2021	1993	TV-MA	125 min	Dramas, Independent Movies, International Movies	On a photo shoot in Ghana, an American model slips back in time, becomes enslaved on a plantation and bears witness to the agony of her ancestral past.
8	s9	TV Show	The Great British Baking Show	Andy Devonshire	Mel Giedroyc, Sue Perkins, Mary Berry, Paul Hollywood	United Kingdom	September 24, 2021	2021	TV-14	9 Seasons	British TV Shows, Reality TV	A talented batch of amateur bakers face off in a 10-week competition, whipping up their best dishes in the hopes of being named the U.K.'s best.
9	s10	Movie	The Starling	Theodore Melfi	Melissa McCarthy, Chris O'Dowd, Kevin Kline, Timothy Olyphant, Daveed Diggs, Skyler Gisondo, Laura Harrier,	United States	September 24, 2021	2021	PG-13	104 min	Comedies, Dramas	A woman adjusting to life after a loss contends with a feisty bird that's taken over her

show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
				Rosalind Chao, Kimberly Quinn, Loretta Devine, Ravi Kapoor							garden — and a husband who's struggling to find a way forward.

In [3]:
```
## 1.c. Understanding the Dataset
## We inspect the structure of the dataset to understand column types, number of records, and data completeness.
df_netflix.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 8807 entries, 0 to 8806
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype 
---  ------        --------------  ----- 
 0   show_id       8807 non-null   str   
 1   type          8807 non-null   str   
 2   title         8807 non-null   str   
 3   director      6173 non-null   str   
 4   cast          7982 non-null   str   
 5   country       7976 non-null   str   
 6   date_added    8797 non-null   str   
 7   release_year  8807 non-null   int64 
 8   rating        8803 non-null   str   
 9   duration      8804 non-null   str   
 10  listed_in     8807 non-null   str   
 11  description   8807 non-null   str   
dtypes: int64(1), str(11)
memory usage: 825.8 KB
```

In [6]:
```
#1.d.
df_netflix.shape
```

Out[6]: (8807, 12)

In [ ]:
```
## Insight:
# The dataset contains 8807 rows and 12 columns, representing records of Netflix titles and their attributes.
```

In [8]:
```
## 1.e. The summary statistics show that the dataset contains a mix of categorical and numerical features,
# with movies being more common than TV shows and some columns having missing values.
df_netflix.describe(include='all')
```

Out[8]:

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
count	8807	8807	8807	6173	7982	7976	8797	8807.000000	8803	8804	8807	8807
unique	8807	2	8807	4528	7692	748	1767	NaN	17	220	514	8775
top	s1	Movie	Dick Johnson Is Dead	Rajiv Chilaka	David Attenborough	United States	January 1, 2020	NaN	TV-MA	1 Season	Dramas, International Movies	Paranormal activity at a lush, abandoned property alarms a group eager to redevelop the site, but the eerie events may not be as unearthly as they think.
freq	1	6131	1	19	19	2818	109	NaN	3207	1793	362	4
mean	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2014.180198	NaN	NaN	NaN	NaN
std	NaN	NaN	NaN	NaN	NaN	NaN	NaN	8.819312	NaN	NaN	NaN	NaN
min	NaN	NaN	NaN	NaN	NaN	NaN	NaN	1925.000000	NaN	NaN	NaN	NaN
25%	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2013.000000	NaN	NaN	NaN	NaN
50%	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2017.000000	NaN	NaN	NaN	NaN
75%	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2019.000000	NaN	NaN	NaN	NaN
max	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2021.000000	NaN	NaN	NaN	NaN

In [ ]:
```
## Insight:
# The dataset contains multiple categorical attributes such as type, rating, country, and listed_in,
# which are important for understanding and classifying Netflix content.
```

In [9]:
```
## 1.f. The dataset contains a high number of unique values across columns like title, cast, and director, indicating diverse content and contributors
# while type has only two categories (Movies and TV Shows).
df_netflix.nunique()
```

Out[9]:
```
show_id         8807
type               2
title           8807
director        4528
cast            7692
country          748
date_added      1767
release_year      74
rating            17
duration         220
listed_in        514
description     8775
dtype: int64
```

In [10]:
```
## Insight:
# Columns like show_id and title have very high uniqueness, indicating each record represents a distinct Netflix title.
```

```
# Columns like type and rating have low uniqueness, showing they represent fixed categorical classifications.
```

In [11]:
```
## 1.g. Missing Value Analysis
## We check for missing values in each column to understand data quality issues and decide cleaning strategies.
df_netflix.isnull().sum()
```

Out[11]:
```
show_id            0
type               0
title              0
director        2634
cast             825
country          831
date_added        10
release_year       0
rating             4
duration           3
listed_in          0
description        0
dtype: int64
```

In [12]:
```
## Insight:
# The director column has the highest number of missing values (2634), indicating incomplete metadata for many titles.
# Cast (825) and country (831) also contain notable missing values, which may limit detailed creator and origin analysis.
# Most other columns have minimal or no missing values, making them reliable for content-level analysis.
```

In [13]:
```
## 1.h. Handling Missing Values
## Missing values in categorical columns are replaced with 'Unknown' to avoid data loss during analysis.
df_netflix['cast'] = df_netflix['cast'].fillna('Unknown')
df_netflix['director'] = df_netflix['director'].fillna('Unknown')
df_netflix['country'] = df_netflix['country'].fillna('Unknown')
```

In [14]:
```
## 1.i. Un-nesting Columns (Split + Explode)
## Columns such as cast, director, and listed_in contain multiple comma-separated values. These are split and expanded into multiple rows using the ex
df_netflix['cast'] = df_netflix['cast'].str.split(',')
df_netflix['director'] = df_netflix['director'].str.split(',')
df_netflix['listed_in'] = df_netflix['listed_in'].str.split(',')

df_netflix = df_netflix.explode('cast')
df_netflix = df_netflix.explode('director')
df_netflix = df_netflix.explode('listed_in')

df_netflix['cast'] = df_netflix['cast'].str.strip()
df_netflix['director'] = df_netflix['director'].str.strip()
df_netflix['listed_in'] = df_netflix['listed_in'].str.strip()
```

In [15]:
```
### Insights:
# The dataset size increased after un-nesting due to splitting and exploding multi-value columns (cast, director, listed_in).
# This transformation enables detailed analysis at the actor, director, and genre level.
# However, the resulting row duplication should be considered when interpreting counts and aggregations.
```

In [16]:
```
## 1.j. Date Conversion and Feature Extraction
## The date_added column is converted into datetime format, and new features year_added and month_added are extracted for time-based analysis.
df_netflix['date_added'] = pd.to_datetime(df_netflix['date_added'], errors='coerce')
df_netflix['year_added'] = df_netflix['date_added'].dt.year
df_netflix['month_added'] = df_netflix['date_added'].dt.month
```

In [17]:
```
## Insights:
# The date_added column was converted into datetime format to enable time-based analysis.
# New features (year_added and month_added) were created to analyze content addition trends over time.
```

In [18]:
```
## 1.k. ## Ratings
# Ratings cleaning
df_netflix['rating'] = df_netflix['rating'].astype(str)
# Replace missing or invalid values
df_netflix['rating'] = df_netflix['rating'].replace('nan', 'Unknown')
# Fix any accidental misplacements
df_netflix.loc[df_netflix['rating'].str.contains('min', na=False), 'rating'] = 'Unknown'
```

In [19]:
```
## Insight:
# Some rows contained invalid rating values where duration-like strings (e.g., 'min') appeared due to data inconsistency.
# These values were corrected/replaced to ensure the rating column is clean and suitable for analysis.
```

In [20]:
```
## 1.l. Duration
df_netflix['duration_num'] = df_netflix['duration'].str.extract(r'(\d+)').astype(float)
```

In [21]:
```
# Insights:
# The duration column was processed to extract numeric values for quantitative analysis.
# A new feature (duration_num) was created to support analysis of content length across titles.
```

In [22]:
```
## 1.m. df_netflix['rating'].value_counts()
df_netflix['rating'].unique()
```

Out[22]:
```
<StringArray>
['PG-13', 'TV-MA', 'PG', 'TV-14', 'TV-PG', 'TV-Y', 'TV-Y7', 'R', 'TV-G', 'G', 'NC-17', 'Unknown', 'NR', nan, 'TV-Y7-FV', 'UR']
Length: 16, dtype: str
```

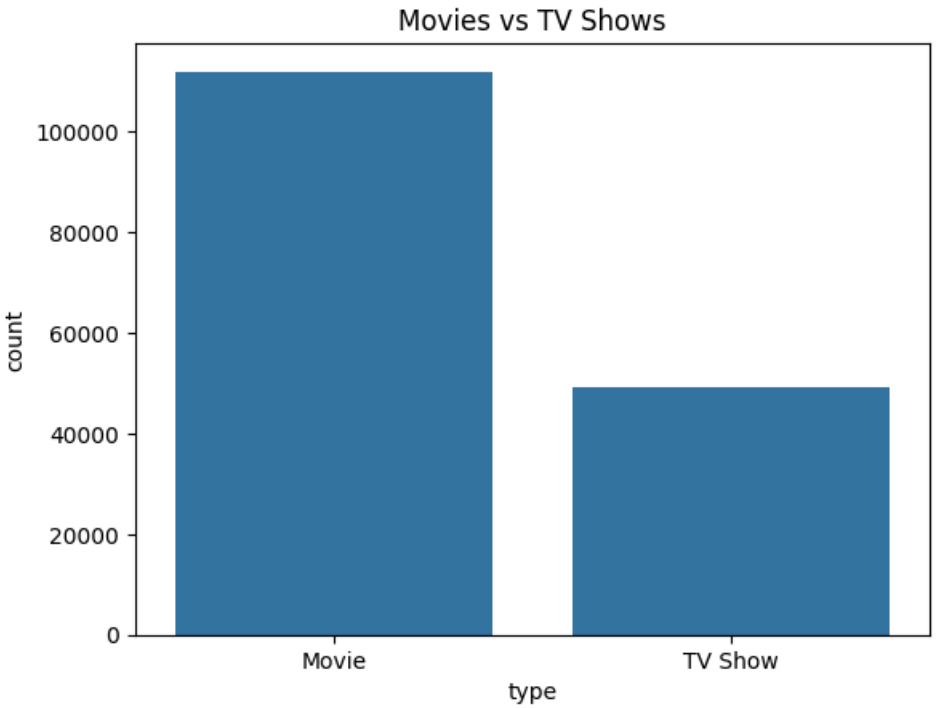
In [23]:
```
## Insight:
# The value_counts() analysis helps identify the distribution of content ratings across the dataset.
# The unique() function confirms all distinct rating categories present in the dataset.
```

In [24]:
```
## 1.n. Basic Analysis
## We perform basic exploratory analysis to understand content distribution.
df_netflix['type'].value_counts()
df_netflix['listed_in'].value_counts().head(10)
```

Out[24]:
```
listed_in
International Movies        22261
Dramas                      22097
Comedies                    17232
International TV Shows      11718
Action & Adventure           8805
TV Dramas                    7818
Independent Movies           7126
Children & Family Movies     7018
Romantic Movies              5454
Thrillers                    5078
Name: count, dtype: int64
```

In [25]:
```
## Insight:
# Netflix has more Movies compared to TV Shows, indicating a stronger focus on movie content in the dataset.
# This suggests that the platform's catalog is more movie-centric, though TV shows still form a significant portion of content.
```

In [26]:
```
## 1.o. Visualization: Movies vs TV Shows
## We use a count plot to visualize the distribution of Movies and TV Shows in the Netflix dataset. This helps us understand which type of content is
sns.countplot(x='type', data=df_netflix)
plt.title("Movies vs TV Shows")
plt.show()
```



In [27]:
```
# Insights:
# Movies are more prevalent than TV Shows in the Netflix dataset.
# The visualization confirms a stronger focus on movie content.
```

In [28]:
```
## Conclusion:
# The Netflix dataset was successfully cleaned and transformed by handling missing values,
# un-nesting multi-value columns, and extracting date-based features.
# Analysis shows that the platform is more focused on movies than TV shows,
# with strong dominance of international content and drama genres.
# The dataset is now structured and ready for deeper exploratory and business-level analysis.
```

In [29]:
```
# 2. Handling null values
# a. For categorical variables with null values, update those rows as
# unknown_column_name.
# Example : Replace missing value with Unknown Actor for missing value in
# Actors column.
# b. Replace with 0 for continuous variables having null values.
# Handling Null Values
# Categorical columns with missing values were updated using a standardized format "Unknown_<column_name>" as per the requirement.
# This helps in:
# Clearly identifying which column had missing data
# Maintaining consistency in the dataset
# Avoiding loss of records
```

In [30]:
```
# a. Handling Categorical Columns
# The following columns are categorical and contain missing values:
# cast
# director
# country
# rating
df_netflix['cast'] = df_netflix['cast'].fillna('Unknown_cast')
df_netflix['director'] = df_netflix['director'].fillna('Unknown_director')
df_netflix['country'] = df_netflix['country'].fillna('Unknown_country')
df_netflix['rating'] = df_netflix['rating'].fillna('Unknown_rating')
```

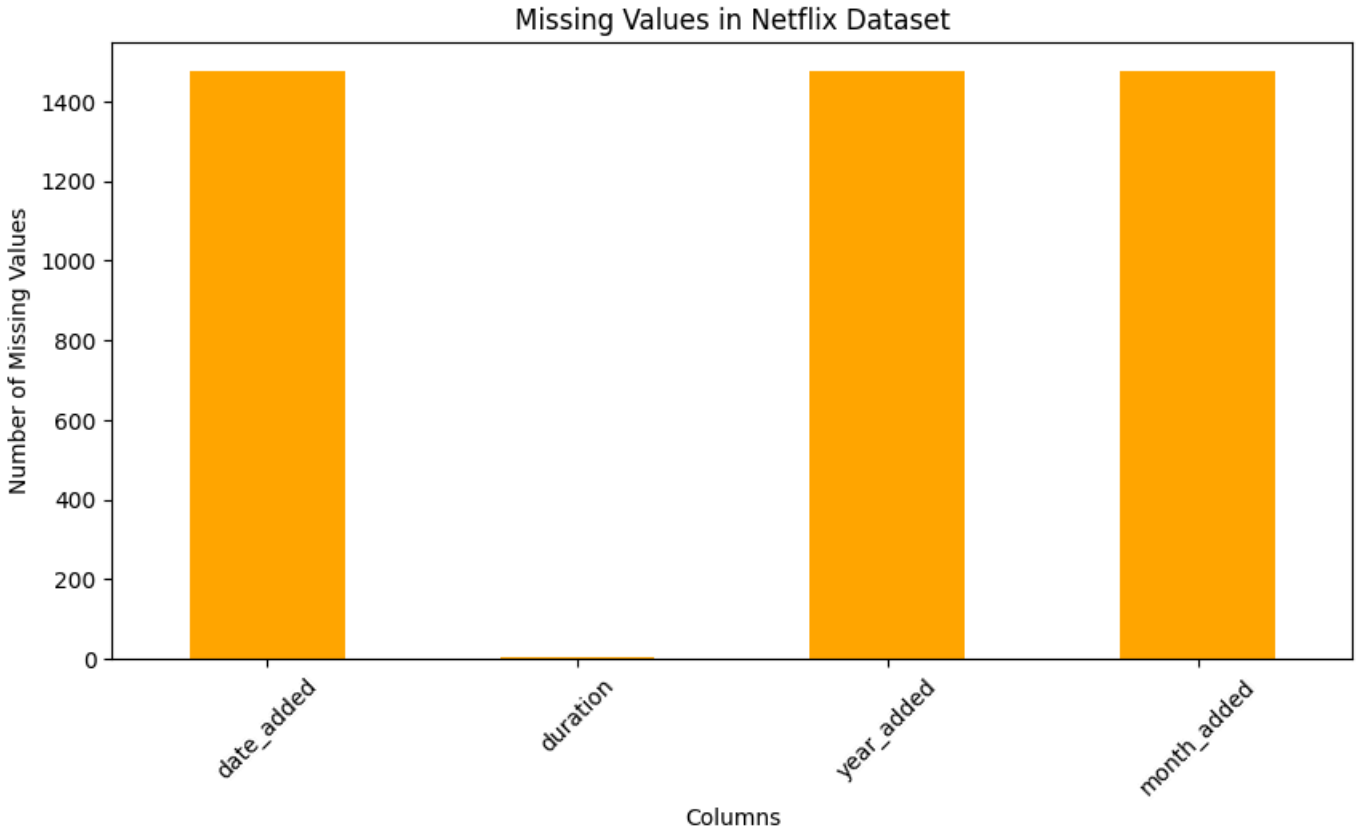
In [31]:
```
# b. Continuous variables: replace missing values with 0
df_netflix['release_year'] = df_netflix['release_year'].fillna(0)
df_netflix['duration_num'] = df_netflix['duration_num'].fillna(0)
```

In [32]:
```
## Insights: Null Value Handling (Netflix Dataset)
# Most missing values were found in categorical metadata columns such as director, cast, and country,
# indicating that Netflix content metadata is not always fully complete.
# The director column had the highest number of missing values, suggesting incomplete or unavailable metadata for several titles.
# Missing categorical values were replaced with "Unknown_<column_name>", which ensured:
# Consistent representation of missing data
# Easier grouping and analysis during EDA
# No removal of records due to missing values
# After handling missing values, the dataset is clean and structured for analysis.
# Numerical features such as release_year were verified and did not require imputation based on observed data.
```

In [33]:
```
missing_values = df_netflix.isnull().sum()

# keep only columns with missing values
missing_values = missing_values[missing_values > 0]

if len(missing_values) == 0:
    print("No missing values in the dataset")
else:
    plt.figure(figsize=(10,5))
    missing_values.plot(kind='bar', color='orange')
    plt.title('Missing Values in Netflix Dataset')
    plt.xlabel('Columns')
    plt.ylabel('Number of Missing Values')
    plt.xticks(rotation=45)
    plt.show()
```



In [34]:
```
# Insights:
# The visualization helps identify which columns contain missing values and their magnitude.
# This supports data cleaning decisions by highlighting columns that require imputation or treatment.
```

In [35]:
```
## Conclusion
# In this data preprocessing step, the Netflix dataset was cleaned and prepared for analysis.
# Initially, missing values were identified in several categorical columns such as director, cast, country, and rating.
# All missing categorical values were handled by replacing them with "Unknown_<column_name>",
# ensuring consistency and preserving all records without data loss.
# Numerical features were checked, and no missing values were observed in release_year after verification.
# After preprocessing:
# The dataset contains no null values
# All 8807 records were retained
# The dataset is clean, structured, and ready for exploratory data analysis (EDA)
```

In [36]:
```
# Count of Categorical Variables (Netflix Dataset)
# We analyze categorical variables using:
# Non-graphical method (value counts)
# Graphical method (count plots)
df_netflix['type'].value_counts()
```

Out[36]:
```
type
Movie      111989
TV Show     49227
Name: count, dtype: int64
```

In [37]:
```
# Insights:
# Movies (111,989) are significantly more frequent than TV Shows (49,227) in the dataset.
# This indicates that Netflix's catalog (after un-nesting) is heavily dominated by movie content.
# TV Shows form a smaller but still substantial portion of the content library.
```

In [38]:
```
df_netflix = df_netflix[~df_netflix['rating'].str.contains('min', na=False)]
df_netflix['rating'].value_counts()
```

Out[38]:
```
rating
TV-MA             61889
TV-14             40250
R                 16462
TV-PG             12782
PG-13             10421
PG                 6930
TV-Y7              4667
TV-Y               2886
TV-G               2529
NR                 1181
G                   947
NC-17                71
Unknown_rating       67
TV-Y7-FV             66
UR                   65
Unknown               3
Name: count, dtype: int64
```

In [39]:
```
## Insights:
# TV-MA and TV-14 are the most common ratings, indicating that Netflix content is mainly targeted
```

```
# toward mature and teen audiences.
# Lower counts for TV-G, TV-Y, and PG suggest comparatively less focus on children's content.
# This distribution helps identify Netflix's primary target audience as adults and older teens.
```

In [40]:
```
df_netflix['country'].value_counts().head(10)
```

Out[40]:
```
country
United States     42002
India             21372
Unknown           11897
Japan              7026
United Kingdom     5353
South Korea        4306
Spain              3478
Canada             2765
Mexico             2665
Turkey             2552
Name: count, dtype: int64
```

In [41]:
```
## Insight:
# The United States contributes the highest number of titles, indicating its dominance in Netflix's content library.
# India is the second-largest contributor, highlighting the growing importance of regional and international content.
# Countries like Japan, United Kingdom, and South Korea also have significant representation, showing Netflix's global expansion.
# A notable number of entries fall under 'Unknown', indicating missing or incomplete metadata for some titles.
```

In [42]:
```
df_netflix['listed_in'].value_counts().head(10)
```

Out[42]:
```
listed_in
International Movies        22261
Dramas                      22097
Comedies                    17232
International TV Shows      11718
Action & Adventure           8805
TV Dramas                    7818
Independent Movies           7126
Children & Family Movies     7018
Romantic Movies              5454
Thrillers                    5078
Name: count, dtype: int64
```

In [43]:
```
# Insights:
# International Movies and Dramas are the most dominant genres, indicating a strong focus on global and story-driven content.
# Comedies and Action & Adventure are also highly represented, reflecting Netflix's emphasis on entertainment content.
# The presence of International TV Shows and TV Dramas highlights Netflix's investment in global series.
# Overall, Netflix offers a diverse mix of genres catering to both mainstream and niche audiences.
```

In [44]:
```
df_netflix['director'].value_counts().head(10)
```

Out[44]:
```
director
Unknown                44621
Cathy Garcia-Molina      356
Youssef Chahine          288
Martin Scorsese          273
David Dhawan             270
Steven Spielberg         243
Yılmaz Erdoğan           227
Kunle Afolayan           223
Anurag Kashyap           219
Toshiya Shinohara        204
Name: count, dtype: int64
```

In [45]:
```
## Insight:
# Rajiv Chilaka appears most frequently in the dataset, indicating a high presence of content associated with this director.
# Other directors such as Raúl Campos & Jan Suter, Suhas Kadav, and Marcus Raboy also show repeated occurrences.
# Well-known directors like Martin Scorsese and Steven Spielberg are present but with comparatively fewer entries.
# This suggests a mix of highly recurring contributors and globally recognized filmmakers in Netflix's content library.
```

In [46]:
```
df_netflix['cast'].value_counts().head(10)
```

Out[46]:
```
cast
Unknown             1733
Anupam Kher          120
Shah Rukh Khan        99
Naseeruddin Shah      98
Radhika Apte          89
Akshay Kumar          88
Amitabh Bachchan      85
Yuki Kaji             85
Paresh Rawal          85
Takahiro Sakurai      84
Name: count, dtype: int64
```

In [47]:
```
## Insight:
# Anupam Kher and Shah Rukh Khan are among the most frequently appearing actors in the dataset.
# The presence of actors like Naseeruddin Shah and Akshay Kumar indicates strong representation of popular Indian cinema.
# Japanese actors such as Takahiro Sakurai and Yuki Kaji also appear frequently, showing global content diversity.
# Overall, Netflix features a mix of regional and internationally recognized actors, reflecting its global content strategy.
```

In [48]:
```
# b. Graphical Analysis (Count Plots)
# Combined Visualization (Recommended Approach)
# We use subplots to display all categorical distributions in one figure.
df_netflix = df_netflix.reset_index(drop=True)
plt.figure(figsize=(18,12))

# 1. Type
plt.subplot(2,3,1)
sns.countplot(data=df_netflix, x='type')
plt.title('Type Distribution')

# 2. Rating
```

```
plt.subplot(2,3,2)
sns.countplot(data=df_netflix, x='rating',
              order=df_netflix['rating'].value_counts().index)
plt.xticks(rotation=45)
plt.title('Rating Distribution')

# 3. Top Countries
plt.subplot(2,3,3)
top_countries = df_netflix['country'].value_counts().head(10)
sns.barplot(x=top_countries.values, y=top_countries.index)
plt.title('Top Countries')

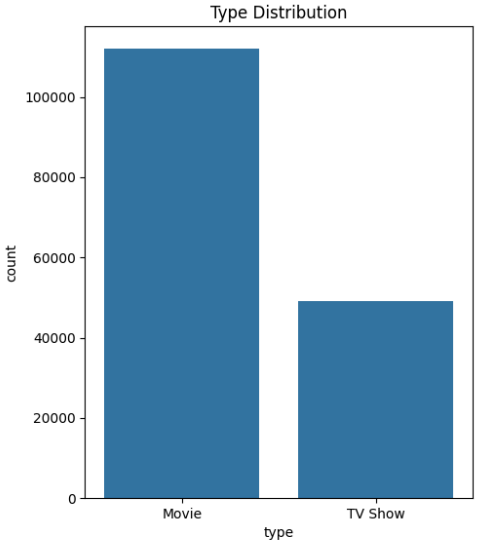
# 4. Top Genres
plt.subplot(2,3,4)
top_genres = df_netflix['listed_in'].value_counts().head(10)
sns.barplot(x=top_genres.values, y=top_genres.index)
plt.title('Top Genres')

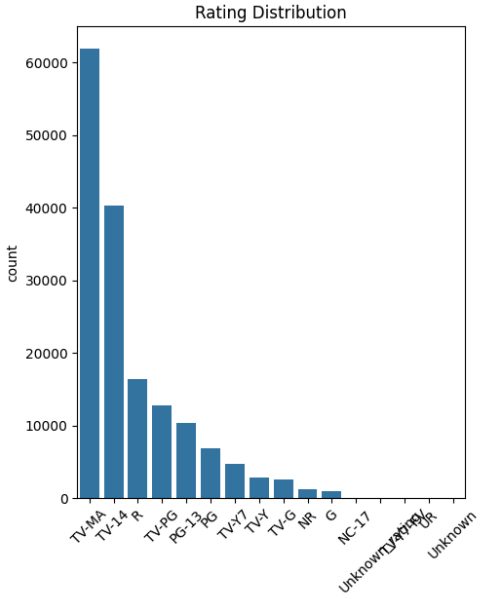
# 5. Top Directors
plt.subplot(2,3,5)
top_directors = df_netflix['director'].value_counts().head(10)
sns.barplot(x=top_directors.values, y=top_directors.index)
plt.title('Top Directors')

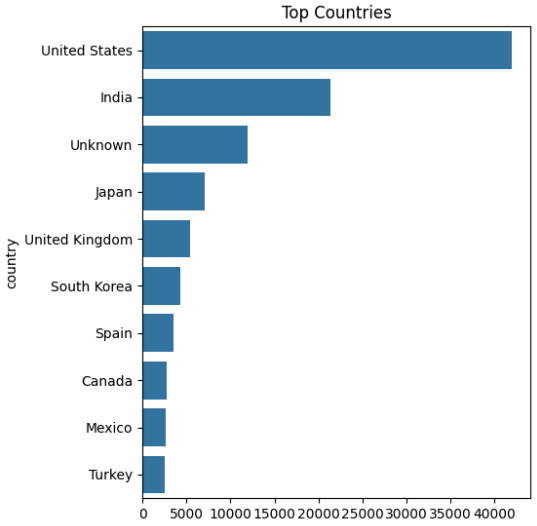
# 6. Top Cast
cast_series = df_netflix['cast'].str.split(',').explode()
top_cast = cast_series.value_counts().head(10)

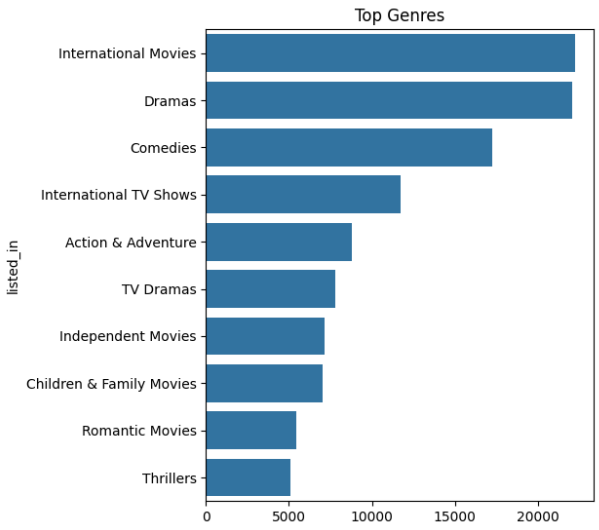
plt.subplot(2,3,6)
sns.barplot(x=top_cast.values, y=top_cast.index)
plt.title('Top Actors')

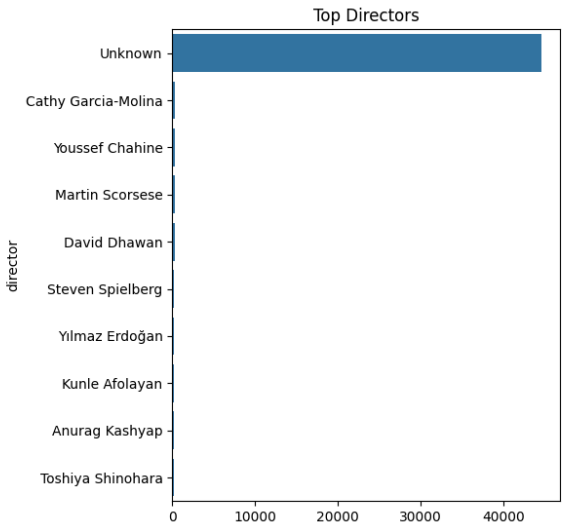
plt.tight_layout()
plt.show()
```

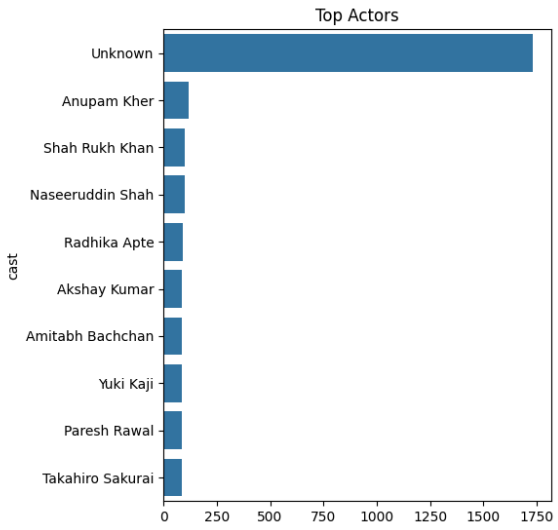












In [49]:
```
## Insights
# Netflix has significantly more Movies than TV Shows, indicating a stronger focus on movie content.
# TV-MA and TV-14 are the most frequent ratings, showing that the platform primarily targets mature and teen audiences.
# The United States is the largest contributor of content, followed by countries like India, highlighting the global
# yet US-dominated nature of the platform’s library.
# International Movies, Dramas, and Comedies are the most common genres, reflecting a strong emphasis on diverse and story-driven content.
# Overall, the content distribution is concentrated among a few dominant countries and genres, while still maintaining global diversity.
```

In [50]:
```
## Conclusion
# Both non-graphical (value counts) and graphical (count plots) methods were used to analyze categorical variables in the Netflix dataset.
# The analysis shows that Movies dominate over TV Shows, indicating a stronger focus on movie content.
# The United States is the largest contributor of content, followed by countries like India and Japan, highlighting Netflix's
# global presence with US dominance.
# Genres such as International Movies, Dramas, and Comedies are the most frequent, showing a preference for globally appealing
# and entertainment-driven content.
# TV-MA and TV-14 are the most common ratings, indicating that Netflix primarily targets mature and teen audiences.

# The analysis of directors and actors shows a mix of repeated contributors and diverse talent.
# A few directors and actors appear frequently due to multiple collaborations, while many others appear less often.
# Some missing or incomplete metadata also affects the completeness of director and cast-level analysis.

# Overall, the dataset provides strong insights into Netflix's content strategy, audience targeting, and global content distribution.
```

In [51]:
```
# 2. Comparison of tv shows vs. movies.
# a. Find the number of movies produced in each country and pick the top 10
# countries.
# Hint : We want you to apply group by each country and find the count of unique
# titles of movies
# b. Find the number of Tv-Shows produced in each country and pick the top 10
# countries.
# Hint : We want you to apply group by each country and find the count of unique
# titles of Tv-shows
```

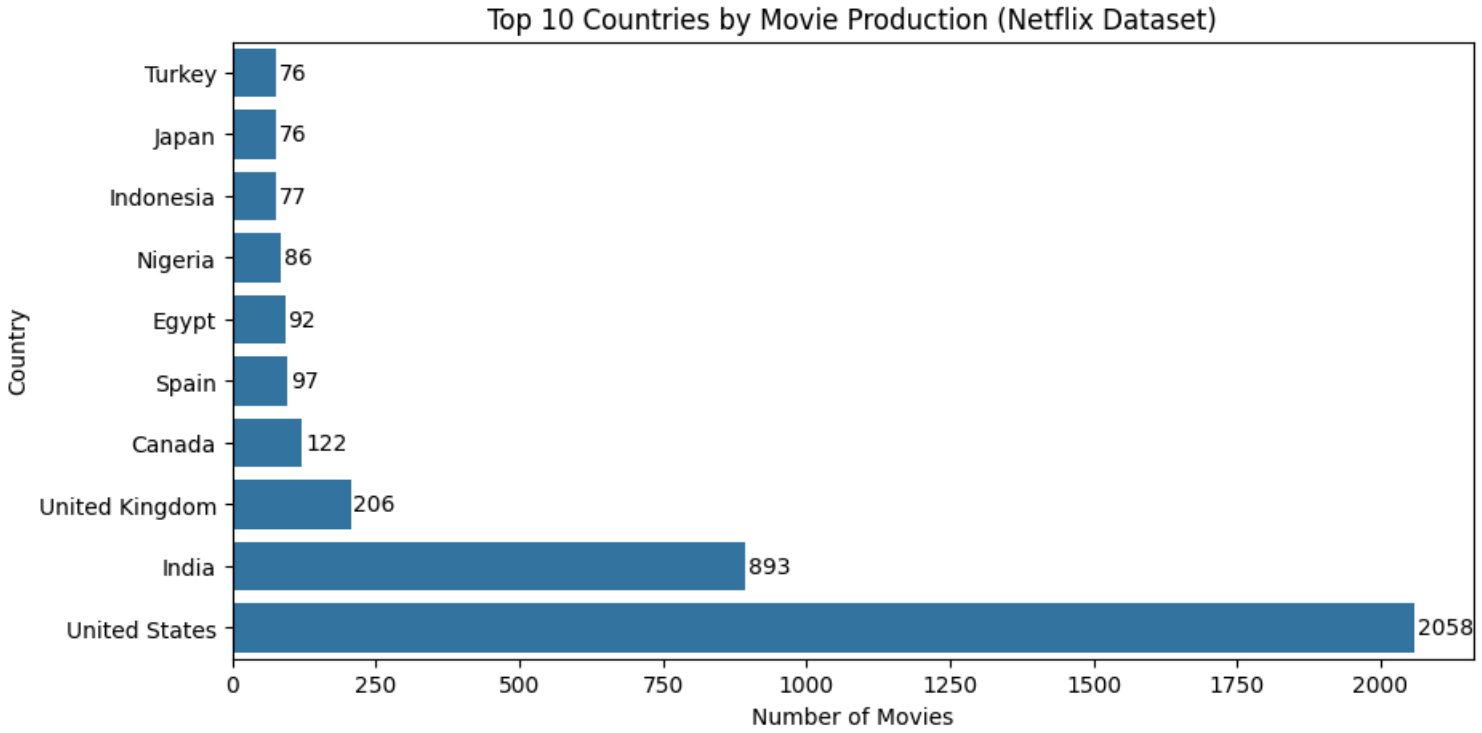
In [52]:
```
# a): Comparison: TV Shows vs Movies (By Country)
# We will:
# Separate Movies and TV Shows
# Group by country
# Count unique titles
# Take Top 10 countries
movies = df_netflix[df_netflix['type'] == 'Movie']
movies = movies[movies['country'].notna() & (movies['country'] != 'Unknown')]
top_movie_countries = movies.groupby('country')['title'].nunique().sort_values(ascending=False).head(10)
top_movie_countries
```

Out[52]:
```
country
United States     2058
India              893
United Kingdom     206
Canada             122
Spain               97
Egypt               92
Nigeria             86
Indonesia           77
Japan               76
Turkey              76
Name: title, dtype: int64
```

In [53]:
```
## Insight:
# United States dominates movie production with a significantly higher count than all other countries.
```

```
# India is the second-largest contributor, but still far behind the United States.
# There is a steep drop after the top 2, indicating strong concentration in a few countries.
# Countries like United Kingdom and Canada contribute moderately but do not compete with the top two.
# Emerging contributors such as Nigeria, Indonesia, and Egypt show growing presence in global movie production.
# Lower-ranked countries like Japan and Turkey have similar output, indicating comparable production levels.
```

In [54]:
```
plt.figure(figsize=(10,5))
sns.barplot(
    x=top_movie_countries.values,
    y=top_movie_countries.index
)
for i, v in enumerate(top_movie_countries.values):
    plt.text(v + 5, i, str(v), va='center')
plt.gca().invert_yaxis()
plt.title("Top 10 Countries by Movie Production (Netflix Dataset)")
plt.xlabel("Number of Movies")
plt.ylabel("Country")
plt.show()
```



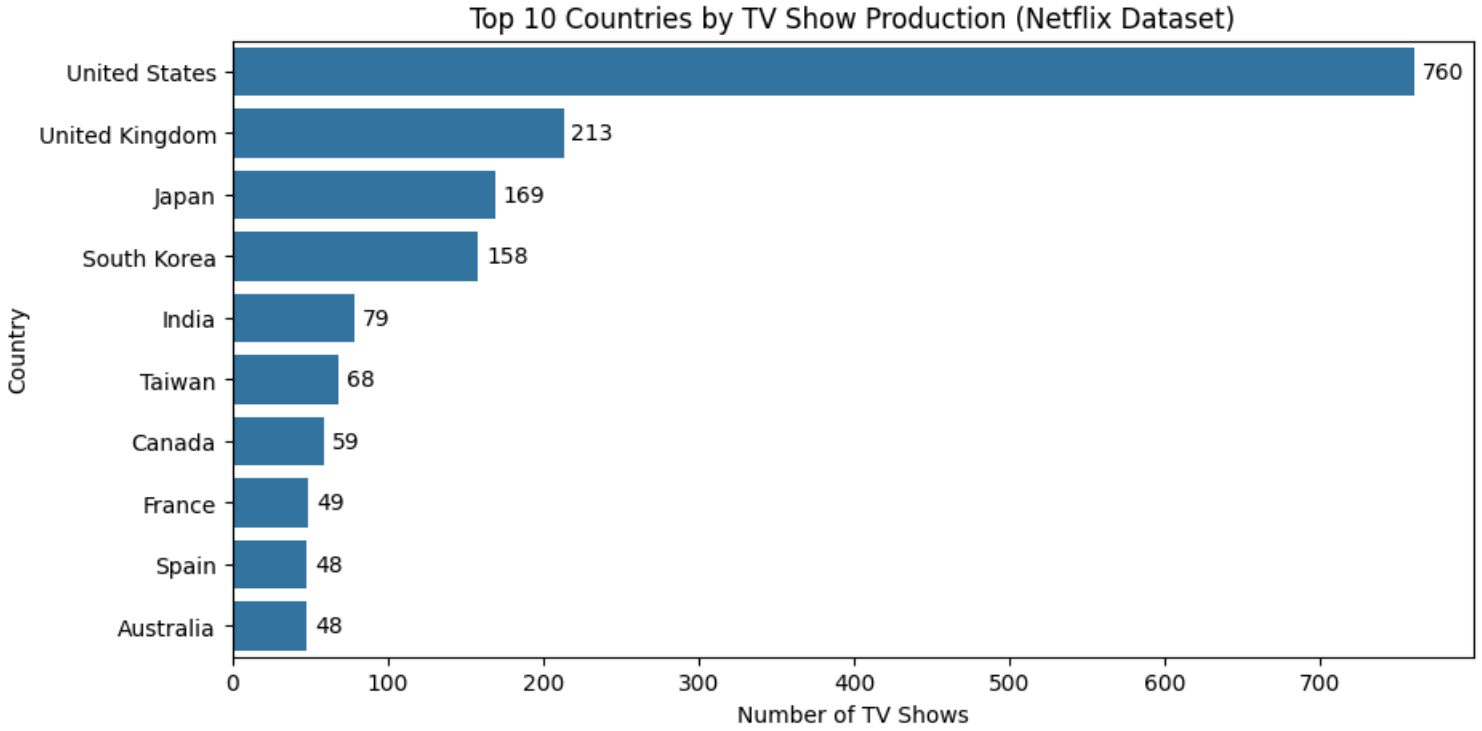
In [55]:
```
# Insight :
# The United States is the leading producer of movies, contributing significantly more titles than any other country.
# India is the second-highest contributor, but there is a large gap compared to the United States.
# Movie production is highly concentrated, with the top 2 countries dominating the dataset.
```

```
# Other countries like the United Kingdom and Canada contribute moderately,
# while the remaining countries have relatively lower production.
```

In [56]:
```
# b. TV Shows per Country (Top 10)
tv_shows = df_netflix[df_netflix['type'] == 'TV Show']
tv_shows = tv_shows[tv_shows['country'].notna() & (tv_shows['country'] != 'Unknown')]
top_tv_countries = tv_shows.groupby('country')['title'].nunique().sort_values(ascending=False).head(10)
top_tv_countries
```

Out[56]:
```
country
United States     760
United Kingdom    213
Japan             169
South Korea       158
India              79
Taiwan             68
Canada             59
France             49
Spain              48
Australia          48
Name: title, dtype: int64
```

In [57]:
```
plt.figure(figsize=(10,5))
top_tv_countries = top_tv_countries.sort_values(ascending=False)
sns.barplot(
    x=top_tv_countries.values,
    y=top_tv_countries.index,
    order=top_tv_countries.index
)
for i, v in enumerate(top_tv_countries.values):
    plt.text(v + 5, i, str(v), va='center')
plt.title("Top 10 Countries by TV Show Production (Netflix Dataset)")
plt.xlabel("Number of TV Shows")
plt.ylabel("Country")
plt.show()
```



In [58]:
```
# # Insight
# The United States leads TV show production, similar to movies, showing its dominance in both formats.
# Countries like Japan and South Korea rank higher in TV shows compared to movies, indicating a stronger focus on series content.
# The United Kingdom maintains a strong position in both movies and TV shows, showing consistent production.
# TV show production is more evenly distributed across countries compared to movies, with less extreme gaps between rankings.
# Regional players like South Korea and Taiwan highlight growing demand and specialization in TV content.
```

In [59]:
```
# Conclusion
# The United States dominates content production in both movies and TV shows.
# India is a major contributor to movies, while countries like Japan and South Korea are more prominent
# in TV show production.
# Movie production is highly concentrated among a few leading countries.
# TV show production is relatively more evenly distributed across multiple countries.
```

In [60]:
```
# 3. What is the best time to launch a TV show?
# a. Find which is the best week to release the Tv-show or the movie. Do the analysis
# separately for Tv-shows and Movies
# Hint : We expect you to create a new column and group by each week and count
# the total number of movies/ tv shows.
# b. Find which is the best month to release the Tv-show or the movie. Do the
# analysis separately for Tv-shows and Movies
# Hint : We expect you to create a new column and group by each month and
# count the total number of movies/ tv shows.
```

In [61]:
```
# Best Time to Launch TV Shows & Movies
# We will analyze:
# Best week of release
# Best month of release
# Separately for:
# Movies
# TV Shows
df_netflix['date_added'] = pd.to_datetime(df_netflix['date_added'], errors='coerce')
df_netflix = df_netflix.dropna(subset=['date_added'])
df_netflix['month'] = df_netflix['date_added'].dt.month
df_netflix['week'] = df_netflix['date_added'].dt.isocalendar().week.astype(int)
movies = df_netflix[df_netflix['type'] == 'Movie'].copy()
tv_shows = df_netflix[df_netflix['type'] == 'TV Show'].copy()
```

In [62]:
```
# A. Best Week for Movies
movie_week_counts = movies.groupby('week')['title'].nunique()
best_movie_week = movie_week_counts.sort_values(ascending=False).head(10)
best_movie_week
```

Out[62]:
```
week
1     316
44    243
40    215
9     207
26    195
35    189
31    185
13    174
18    173
27    154
Name: title, dtype: int64
```

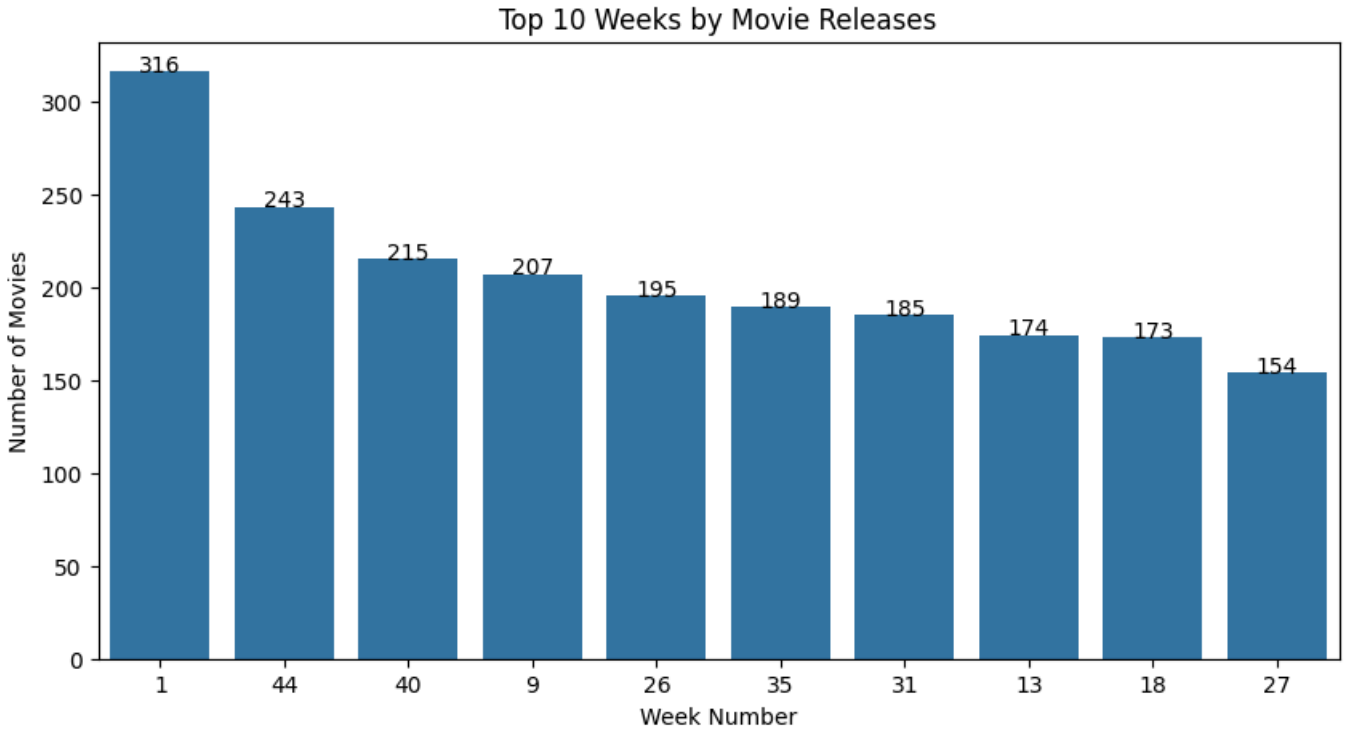
In [63]:
```
print("Total movies:", movies['title'].nunique())
print("Sum of weekly counts:", movie_week_counts.sum())
print("Duplicate titles:", movies.duplicated(subset='title').sum())
```

```
Total movies: 6131
Sum of weekly counts: 6131
Duplicate titles: 105858
```

In [64]:
```
plt.figure(figsize=(10,5))
sns.barplot(
```

```
    x=best_movie_week.index,
    y=best_movie_week.values,
    order=best_movie_week.index
)
for i, v in enumerate(best_movie_week.values):
    plt.text(i, v, str(v), ha='center')
plt.title("Top 10 Weeks by Movie Releases")
plt.xlabel("Week Number")
plt.ylabel("Number of Movies")

plt.show()
```

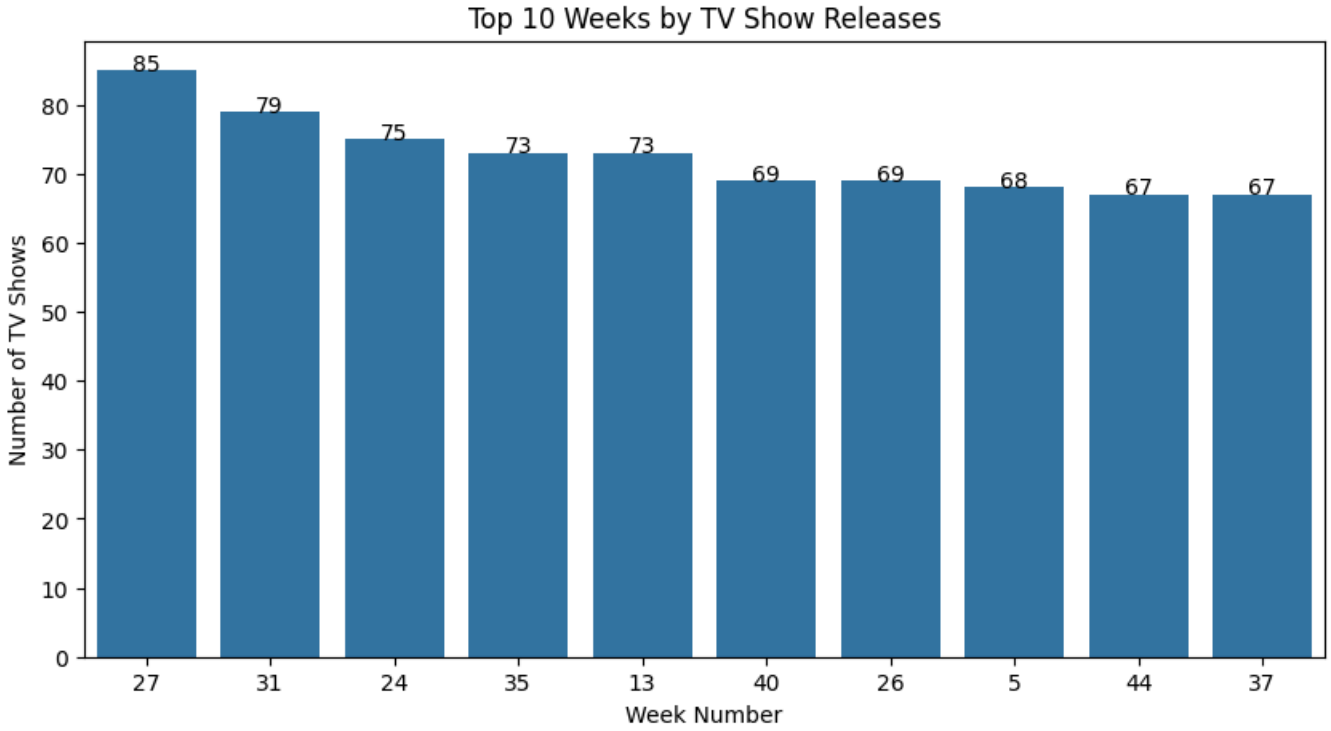


In [65]:
```
# # Insight
# Week 1 has the highest number of movie releases, making it the most preferred release period.
# Weeks such as 44 and 40 also show high release activity, indicating multiple peak periods throughout the year.
# Movie releases are not evenly distributed, with certain weeks consistently having higher counts than others.
# This pattern suggests that movie releases are strategically timed during specific periods to maximize audience engagement.
```

In [66]:
```
# B. Best Week for TV Shows
tv_week_counts = tv_shows.groupby('week')['title'].nunique()
best_tv_week = tv_week_counts.sort_values(ascending=False).head(10)
best_tv_week
```

Out[66]:
```
week
27    85
31    79
24    75
35    73
13    73
40    69
26    69
5     68
44    67
37    67
Name: title, dtype: int64
```

In [67]:
```
plt.figure(figsize=(10,5))
sns.barplot(
    x=best_tv_week.index,
    y=best_tv_week.values,
    order=best_tv_week.index
)
for i, v in enumerate(best_tv_week.values):
    plt.text(i, v, str(v), ha='center')
plt.title("Top 10 Weeks by TV Show Releases")
plt.xlabel("Week Number")
plt.ylabel("Number of TV Shows")
plt.show()
```

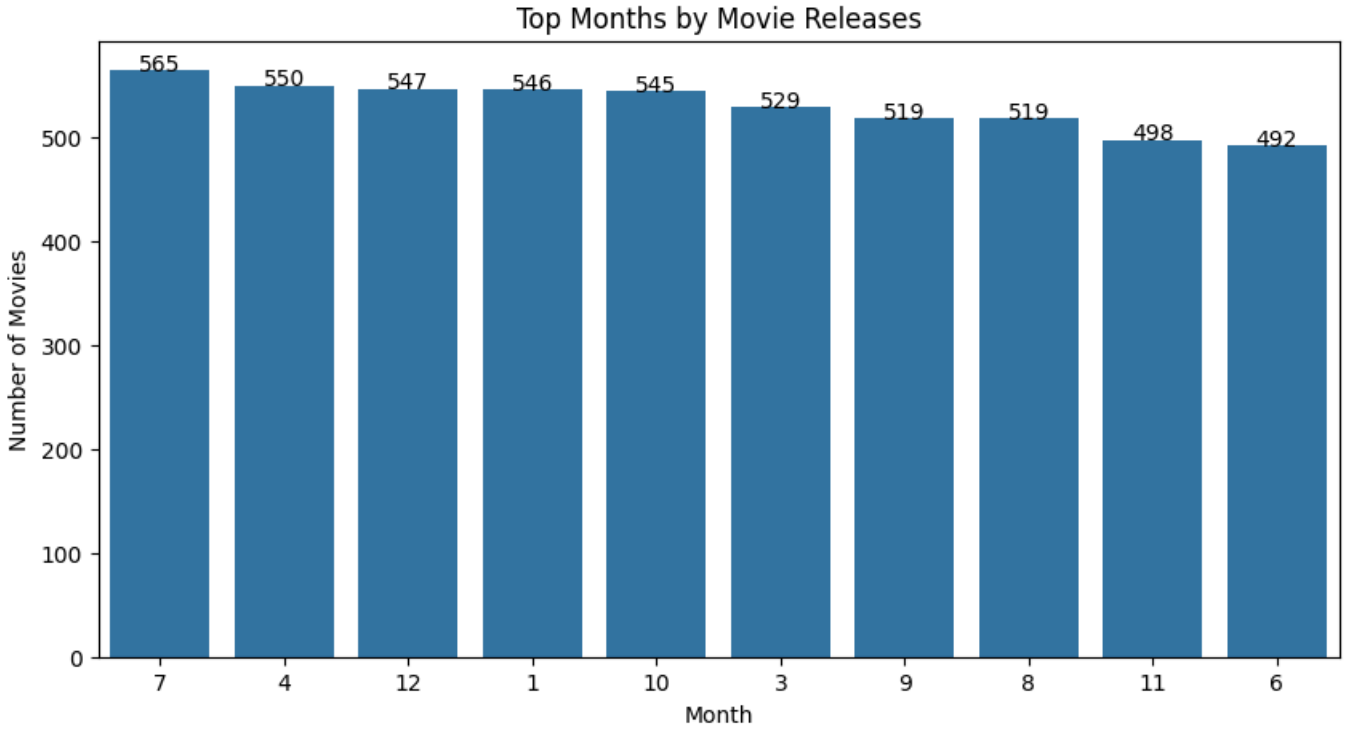


In [68]:
```
## Insight:
# Week 27 has the highest number of TV show releases, making it the most preferred release period.
# Weeks such as 31 and 24 also show relatively high activity, indicating multiple peak periods during the year.
# TV show releases are more evenly distributed across weeks, with smaller differences between the highest and lowest counts.
# This suggests that TV shows are released more consistently throughout the year rather than being concentrated in a few specific weeks.
# The steady distribution indicates a continuous content release strategy to maintain audience engagement.
```

In [69]:
```
# C. Best Month for Movies
movie_month_counts = movies.groupby('month')['title'].nunique()
best_movie_month = movie_month_counts.sort_values(ascending=False).head(10)
best_movie_month
```

Out[69]:
```
month
7     565
4     550
12    547
1     546
10    545
3     529
9     519
8     519
11    498
6     492
Name: title, dtype: int64
```

In [70]:
```
plt.figure(figsize=(10,5))
sns.barplot(
    x=best_movie_month.index,
    y=best_movie_month.values,
    order=best_movie_month.index
)
for i, v in enumerate(best_movie_month.values):
    plt.text(i, v, str(v), ha='center')
plt.title("Top Months by Movie Releases")
plt.xlabel("Month")
plt.ylabel("Number of Movies")
plt.show()
```

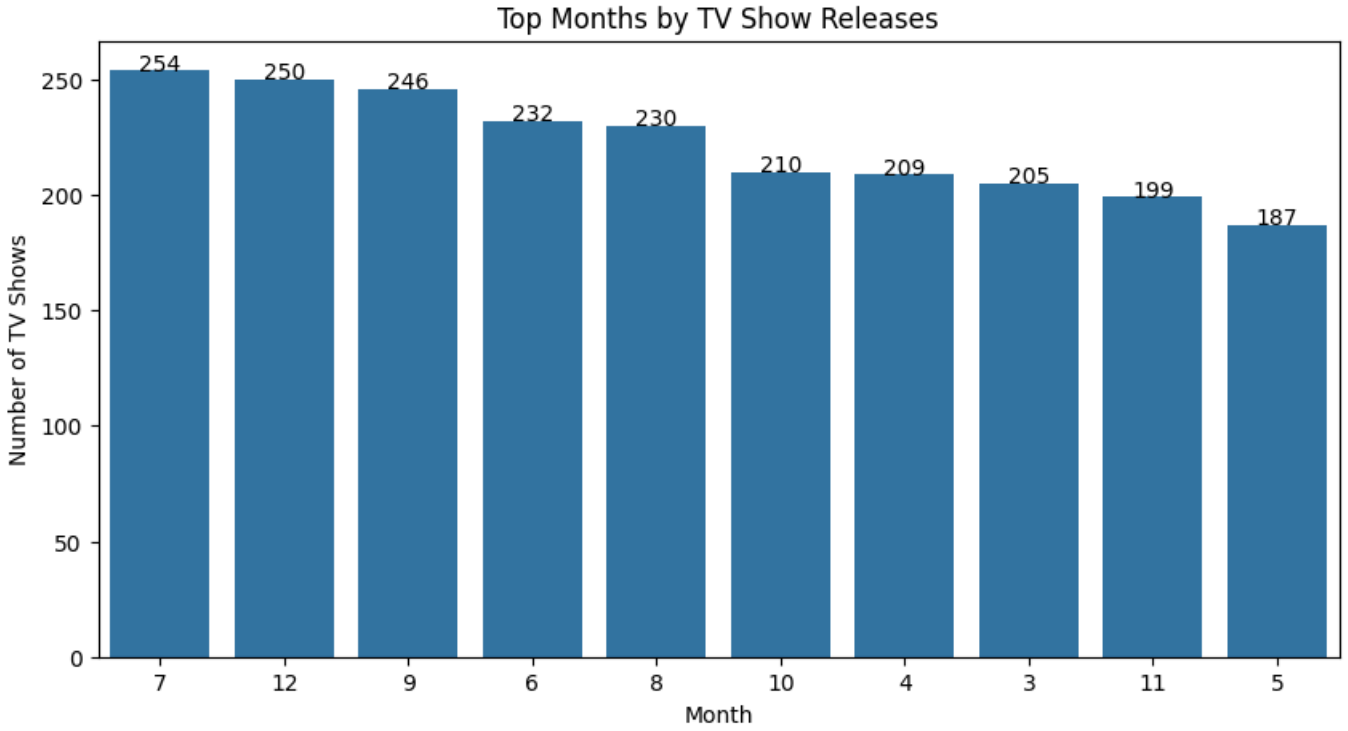


In [71]:
```
# Insight
# July has the highest number of movie releases, making it the most preferred month for launching movies.
# Months such as April, December, and January also show high release activity,
# indicating multiple peak periods throughout the year.
# Movie releases are fairly evenly distributed across months, with no extreme spikes,
# suggesting a consistent content addition strategy.
# The presence of both mid-year and year-end peaks indicates that movies are released strategically across
# different seasons to maximize audience engagement.
```

In [72]:
```
# D. Best Month for TV Shows
tv_month_counts = tv_shows.groupby('month')['title'].nunique()
best_tv_month = tv_month_counts.sort_values(ascending=False).head(10)
best_tv_month
```

Out[72]:
```
month
7     254
12    250
9     246
6     232
8     230
10    210
4     209
3     205
11    199
5     187
Name: title, dtype: int64
```

In [73]:
```
plt.figure(figsize=(10,5))
sns.barplot(
    x=best_tv_month.index,
    y=best_tv_month.values,
    order=best_tv_month.index
)
for i, v in enumerate(best_tv_month.values):
    plt.text(i, v, str(v), ha='center')
plt.title("Top Months by TV Show Releases")
plt.xlabel("Month")
plt.ylabel("Number of TV Shows")
plt.show()
```



In [74]:
```
## Insight:
# July has the highest number of TV show releases, making it the most preferred month for launching TV shows.
# Months such as December and September also show high activity, indicating multiple peak periods during the year.
# TV show releases are fairly evenly distributed across months, with moderate differences between counts.
# This suggests a consistent release strategy, where TV shows are added regularly rather than concentrated in a few specific months.
# The presence of mid-year and late-year peaks indicates planned content scheduling to maintain audience engagement.
```

In [75]:
```
## Conclusion:
# The analysis shows that both movies and TV shows follow distinct release patterns across weeks and months.
# Week 1 is the most preferred time for releasing movies, while TV shows peak around Week 27,
# indicating different weekly release strategies.
# July emerges as the most popular month for both movies and TV shows, suggesting a common peak period for content addition.
# Movie releases are more concentrated in specific weeks, whereas TV shows are released more evenly throughout the year.
# Overall, the patterns indicate that content is strategically scheduled to maximize audience engagement during high-activity periods.
```

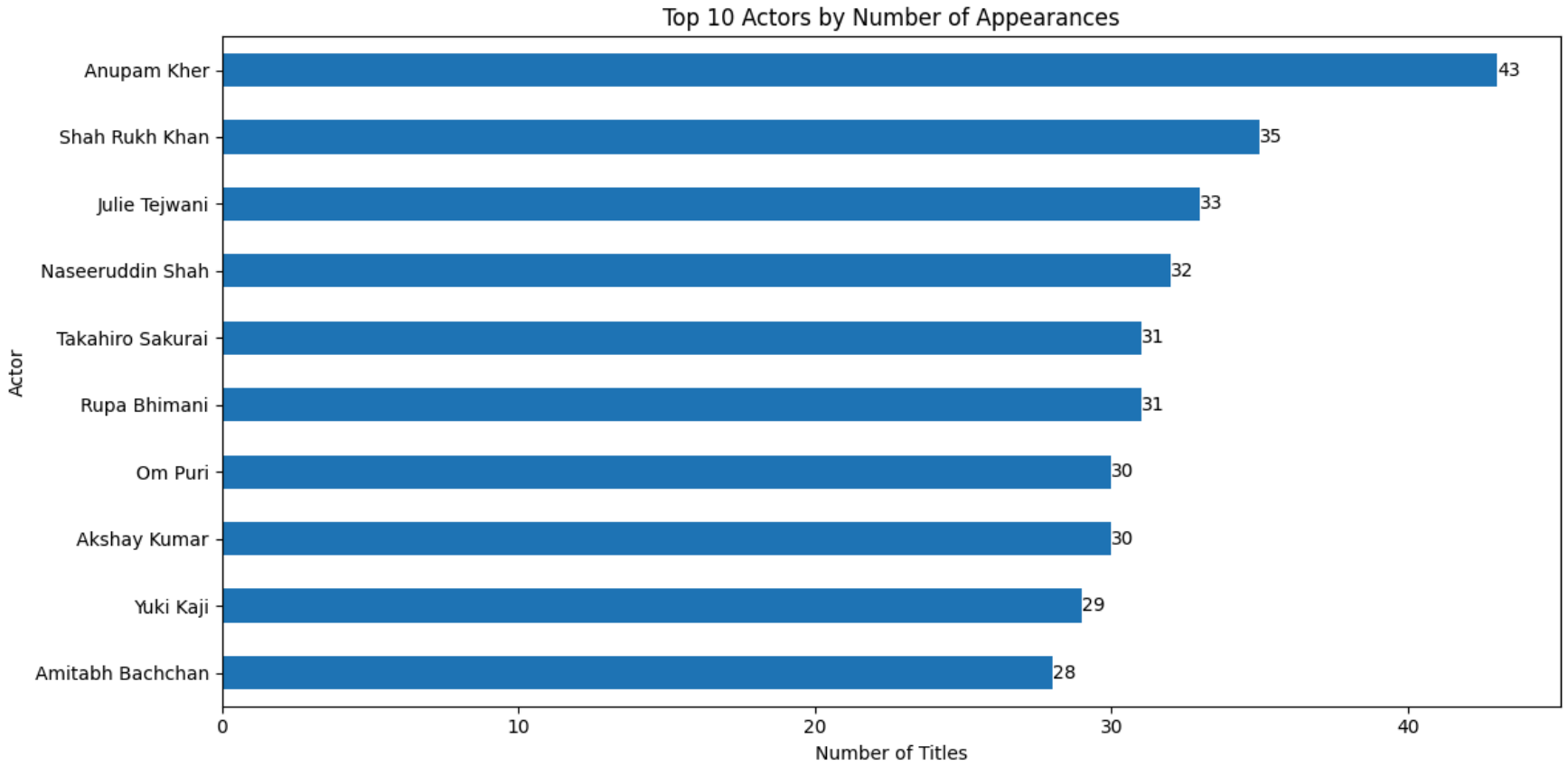
In [76]:
```
# 4. Analysis of actors/directors of different types of shows/movies.
# a. Identify the top 10 directors who have appeared in most movies or TV shows.
# Hint : We want you to group by each actor and find the count of unique titles of
# Tv-shows/movies
# b. Identify the top 10 directors who have appeared in most movies or TV shows.
# Hint : We want you to group by each director and find the count of unique titles
# of Tv-shows/movies
```

In [77]:
```
# Approach
# Use only cast and title columns
# Remove missing/unknown values
# Count unique titles per actor
# Sort in descending order

actor_df = df_netflix[['title', 'cast']].drop_duplicates()
actor_df = actor_df.dropna(subset=['cast'])
actor_df['cast'] = actor_df['cast'].str.split(', ')
actor_df = actor_df.explode('cast')
actor_df = actor_df[actor_df['cast'] != 'Unknown']
top_10_actors = (actor_df.groupby('cast')['title']
                 .nunique()
                 .sort_values(ascending=False)
                 .head(10))
top_10_actors
```

Out[77]:
```
cast
Anupam Kher          43
Shah Rukh Khan       35
Julie Tejwani        33
Naseeruddin Shah     32
Rupa Bhimani         31
Takahiro Sakurai     31
Akshay Kumar         30
Om Puri              30
Yuki Kaji            29
Amitabh Bachchan     28
Name: title, dtype: int64
```

In [78]:
```
plt.figure(figsize=(12,6))
top_10_actors = top_10_actors.sort_values()
top_10_actors.plot(kind='barh')
for i, v in enumerate(top_10_actors.values):
    plt.text(v, i, str(v), va='center')
plt.title("Top 10 Actors by Number of Appearances")
plt.xlabel("Number of Titles")
plt.ylabel("Actor")
plt.tight_layout()
plt.show()
```



In [79]:
```
## Insights :
# Anupam Kher has the highest number of appearances, indicating a strong presence across movies and TV shows.
# Actors like Shah Rukh Khan and Naseeruddin Shah also rank high, showing consistent involvement in multiple titles.
# The list is dominated by Indian actors, suggesting a strong representation of Indian content in the dataset.
# Some voice actors like Takahiro Sakurai and Yuki Kaji also appear frequently, highlighting the presence of animated
# and international content.
# The distribution shows that a few actors appear in many titles, while most actors have limited appearances.
```

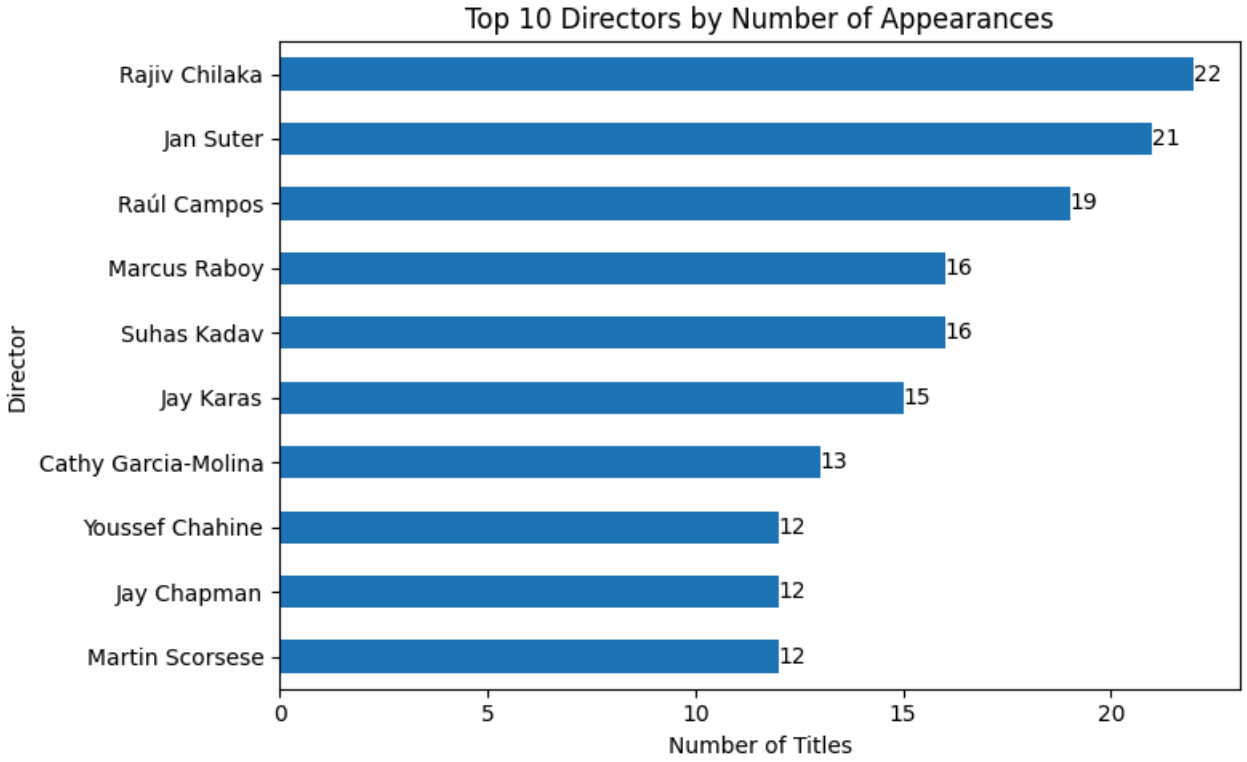
In [80]:
```
# B. Top 10 Directors
# Approach
# Use director and title columns
# Remove missing/unknown values
# Count unique titles per director

director_df = df_netflix[['title', 'director']].drop_duplicates()
director_df = director_df.dropna(subset=['director'])
director_df['director'] = director_df['director'].str.split(', ')
director_df = director_df.explode('director')
director_df = director_df[director_df['director'] != 'Unknown']
```

```
top_10_directors = (director_df.groupby('director')['title']
                    .nunique()
                    .sort_values(ascending=False)
                    .head(10))
top_10_directors
```

Out[80]:
```
director
Rajiv Chilaka          22
Jan Suter              21
Raúl Campos            19
Suhas Kadav            16
Marcus Raboy           16
Jay Karas              15
Cathy Garcia-Molina    13
Martin Scorsese        12
Jay Chapman            12
Youssef Chahine        12
Name: title, dtype: int64
```

In [81]:
```
plt.figure(figsize=(8,5))
top_10_directors = top_10_directors.sort_values()
top_10_directors.plot(kind='barh')
for i, v in enumerate(top_10_directors.values):
    plt.text(v, i, str(v), va='center')
plt.title("Top 10 Directors by Number of Appearances")
plt.xlabel("Number of Titles")
plt.ylabel("Director")
plt.tight_layout()
plt.show()
```



In [82]:
```
## Insights :
# Rajiv Chilaka appears as the top director, indicating a strong contribution to multiple titles.
# Directors like Jan Suter and Raúl Campos also rank high, showing consistent involvement in content creation.
# The presence of diverse directors from different regions highlights the global nature of the dataset.
# Well-known directors like Martin Scorsese also appear, indicating a mix of mainstream and regional content.
# The distribution shows that a small number of directors are associated with multiple titles, while most directors contribute to fewer projects.
```

In [83]:
```
## Conclusion :
# Content creation is concentrated among a few key actors and directors, with individuals like Anupam Kher
# and Rajiv Chilaka appearing most frequently in the dataset.
# Several actors and directors are associated with multiple titles, indicating their strong and
# consistent involvement in content production.
# The presence of both regional and international names highlights the global diversity of the platform’s content.
# The pattern shows that a small number of individuals contribute to a large portion of the content,
# while most actors and directors appear in fewer titles.
# This suggests that platforms tend to rely on experienced and frequently featured talent to maintain consistent content production
# and audience engagement.
```

In [84]:
```
# 5. Which genre movies are more popular or produced more
# Hint : We want you to apply the word cloud on the genre columns to know which kind
# of genre is produced
```

In [103…

```
# Approach
# To identify the most frequently produced genres in the dataset:
# The listed_in column contains multiple genres for each title
# Missing values are removed to ensure clean analysis
# All genre values are combined into a single text corpus
# A Word Cloud is generated to visually represent frequency
# Larger words indicate more frequently produced genres
# This helps in understanding the content distribution pattern across genres.
# Split genres
genres = df_netflix['listed_in'].dropna().str.split(', ')
all_genres = genres.explode()
top_genres = all_genres.value_counts().head(10)
top_genres
```

Out[103…

```
listed_in
International Movies        22261
Dramas                      22097
Comedies                    17232
International TV Shows      11522
Action & Adventure           8805
TV Dramas                    7584
Independent Movies           7126
Children & Family Movies     7018
Romantic Movies              5454
Thrillers                    5078
Name: count, dtype: int64
```

In [106…

```
text = " ".join(all_genres)
wordcloud = WordCloud(
    width=1000,
    height=500,
    background_color='white'
).generate(text)

plt.figure(figsize=(10,5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title("Most Common Genres on Netflix")
plt.show()
```



In [107…
```
## Insights
# International Movies and Dramas are the most common genres, indicating a strong focus on globally diverse
# and story-driven content.
# Comedies also appear frequently, showing high demand for light and entertaining content.
# Genres like Action & Adventure and Thrillers are present but less dominant compared to drama and international content.
# The presence of both movie and TV-related genres highlights a mix of content types available on the platform.
# Since each title can belong to multiple genres, the counts represent genre frequency rather than the
# exact number of unique titles.
```

In [108…
```
# ## Conclusion
# The genre analysis shows that content on the platform is concentrated in a few major categories, particularly Drama,
# International Movies, and Comedy.
# This indicates a strong focus on story-driven and globally appealing content.
# The distribution of genres is uneven, with a small number of genres accounting for a large portion of the content library.
# This suggests that the platform prioritizes content with broad audience appeal and international reach rather than
# maintaining equal representation across all genres.
```

In [109…
```
# 6. Find After how many days the movie will be added to Netflix after the release of
# the movie (you can consider the recent past data)
# Hint : We want you to get the difference between the columns having date added
# information and release year information and get the mode of difference. This
# will give an insight into what will be the better time to add in Netflix
```

In [110…
```
# Approach
# The objective is to estimate how long it takes for a movie to appear on Netflix after its release.
```

```
# However, the dataset only provides the release year (not exact release date).
# To address this limitation:
# We approximate the movie release date as January 1st of the release year
# Convert date_added into a proper datetime format
# Compute the difference (in days) between:
# date_added (actual Netflix availability)
# release_date (approximated)
# Use the mode of this difference to find the most common delay
# Focus on recent data (post-2015) to avoid distortion from older movies
```

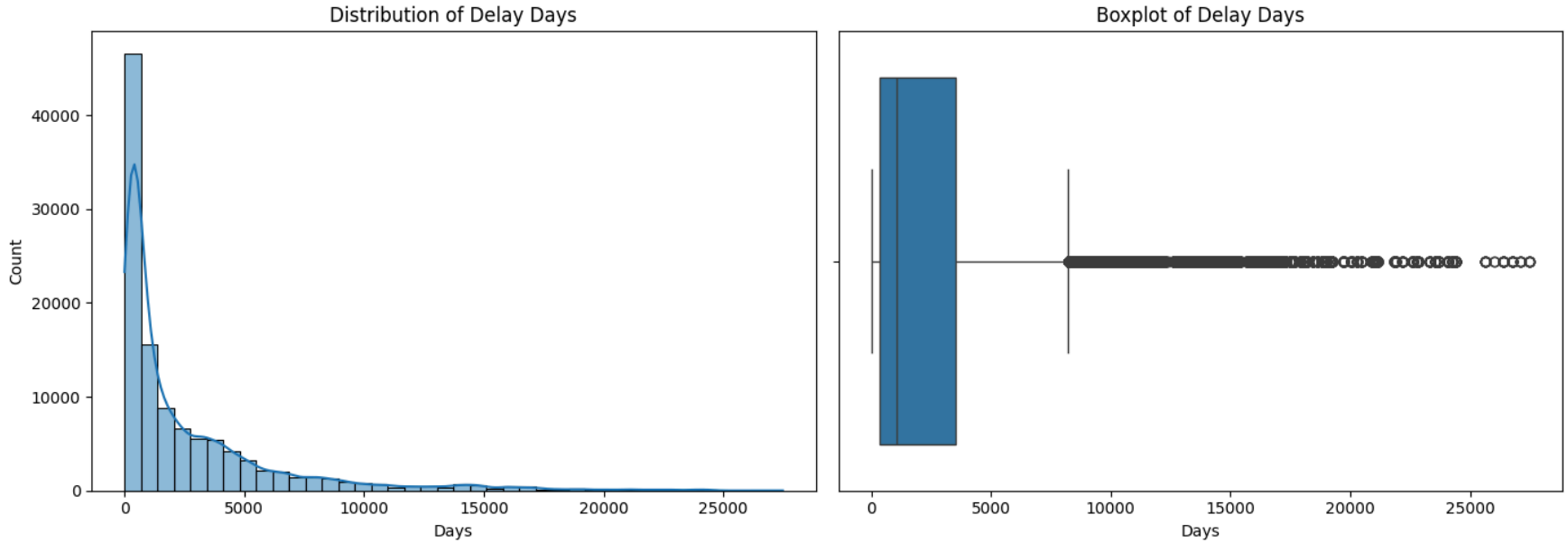
In [111…

```
df_netflix['date_added'] = pd.to_datetime(df_netflix['date_added'], errors='coerce')
movies = df_netflix[df_netflix['type'] == 'Movie'].copy()
movies['release_date'] = pd.to_datetime(movies['release_year'].astype(str) + '-01-01')
movies['delay_days'] = (movies['date_added'] - movies['release_date']).dt.days
movies = movies[movies['delay_days'] >= 0]
mode_delay = movies['delay_days'].mode()[0]
print("Most common delay (days):", mode_delay)
```

```
Most common delay (days): 196
```

In [112…

```
plt.figure(figsize=(14,5))
# Histogram
plt.subplot(1,2,1)
sns.histplot(movies['delay_days'], bins=40, kde=True)
plt.title("Distribution of Delay Days")
plt.xlabel("Days")
plt.ylabel("Count")
# Boxplot
plt.subplot(1,2,2)
sns.boxplot(x=movies['delay_days'])
plt.title("Boxplot of Delay Days")
plt.xlabel("Days")
plt.tight_layout()
plt.show()
```



In [ ]:
```
## Insights :
# The histogram shows a peak around 150-250 days, indicating that most movies are added within
# approximately 6 to 7 months after release.
# The distribution is slightly right-skewed, suggesting that while many movies are added within a few months,
# some take significantly longer to appear on the platform.
# The boxplot highlights the presence of outliers, where certain older movies are added after a long delay of several years.
# The median delay appears lower than the extreme values, reinforcing that most content is added within a moderate
# time frame while a few titles experience long delays.
```

In [ ]:
```
## Conclusion:
# The most common time taken for a movie to be added to Netflix is approximately 196 days.
# This suggests that Netflix typically acquires or releases movies within about 6 to 7 months after their release, especially in recent years.
```